Jenkins CI/CD Pipeline Exercise

End-to-End Hands-on Guide for Java Maven Project using GitHub & Pipeline as Code

PIPELINE ARCHITECTURE & FLOW

1. GitHub Repo (Push / Commit) → **2. Checkout** (Jenkins Pipeline) → **3. Build & Test** (mvn clean package) → **4. Deployment** (Target Server)

STEP 1 Set Up Your GitHub Repository

Prerequisite setup for the Maven project structure

Create or fork a simple Java Maven project on GitHub. Ensure your repository has the standard directory layout:

```
my-java-app/
├── pom.xml
└── src/
    ├── main/java/com/example/App.java
    └── test/java/com/example/AppTest.java
```

Note: Copy your repository URL (e.g., `https://github.com/your-username/my-java-app.git`).

STEP 2 Configure Global Tools in Jenkins

Setting up JDK and Maven paths in Jenkins Administration

Navigate to **Dashboard > Manage Jenkins > Tools**.

Jenkins Dashboard Interface Preview

JDK Installations

Name: `JDK-17` | [✓] Install automatically

Maven Installations

Name: `Maven-3.9` | [✓] Install automatically

- **JDK Setup:** Click *Add JDK*, set Name to `JDK-17`, and choose automatic installation.
- **Maven Setup:** Click *Add Maven*, set Name to `Maven-3.9`, and choose automatic installation.
- Click **Save** to persist changes.

STEP 3 Add Jenkinsfile to Your GitHub Repository

Defining CI/CD stages using Declarative Pipeline syntax

Create a file named `Jenkinsfile` at the root level of your repository and commit the following code:

```
pipeline {
    agent any

    tools {
        maven 'Maven-3.9'
        jdk 'JDK-17'
    }

    stages {
        stage('1. Checkout Code') {
            steps {
                echo 'Fetching source code from GitHub...'
                checkout scm
            }
        }

        stage('2. Build') {
            steps {
                echo 'Compiling the Java application...'
                sh 'mvn clean compile'
            }
        }

        stage('3. Unit Test') {
            steps {
                echo 'Running unit tests...'
                sh 'mvn test'
            }
            post {
                always {
                    junit '**/target/surefire-reports/*.xml'
                }
            }
        }

        stage('4. Package') {
            steps {
                echo 'Packaging application into JAR/WAR...'
                sh 'mvn package -DskipTests'
            }
        }

        stage('5. Deploy') {
            steps {
                echo 'Deploying application artifact...'
                sh '''
                    echo "Deploying target file to destination server..."
                    cp target/*.jar /tmp/deployed-app.jar || true
                    echo "Deployment completed successfully."
                '''
            }
        }
    }
```

```
    post {
        success { echo 'Pipeline completed successfully!' }
        failure { echo 'Pipeline failed. Check the logs for details.' }
    }
}
```

STEP 4 Create and Configure the Jenkins Job

Linking Jenkins Pipeline job to GitHub SCM

1. Go to Jenkins Dashboard and click **New Item**.
2. Enter item name: `java-mvn-cicd-pipeline` and select **Pipeline**.
3. In the **Pipeline** section, configure as follows:

Jenkins Pipeline Job Configuration Screenshot Simulation

Definition
`Pipeline script from SCM`

SCM
`Git`

Repository URL
`https://github.com/your-username/my-java-app.git`

Branch Specifier
`*/main`

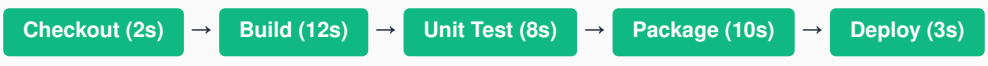
Script Path
`Jenkinsfile`

STEP 5 Run and Validate the Pipeline

Executing build and verifying visual output

1. Click **Build Now** from the left navigation panel.
2. Monitor the execution through **Stage View**:

Jenkins Stage View Visual Output



3. Inspect test reports and build output under **Console Output** for complete logs.

Key Pipeline Directives Reference

Directive	Description & Purpose
`tools { ... }`	Binds predefined system JDK and Maven versions dynamically into execution PATH.
`checkout scm`	Fetches source code automatically from the configured Git repository.
`post { junit ... }`	Parses JUnit XML report files and publishes visual test charts in Jenkins.
`post { success / failure }`	Executes actions or notifications based on overall pipeline status.